UNIVERSITY OF CAMBRIDGE

MPhil in Data Intensive Science

Lent Term 2026

C2 High Performance Computing Module Coursework

The coursework will be submitted via a GitLab repository that will be created for you. You should place all your code and your report in this repository. You will be provided access to the repository ***until 23:59 on Wednesday 1st April 2026****. After this you will lose access which will constitute submission of your work.*

You are advised to keep a clone of your repository should you wish to keep a copy of your work.

You must submit a pdf report named `report.pdf` *in the folder* `report/` *. This must be less than 3000 words in length; it could be much shorter.*

You are reminded that all use of auto-generation tools, for both the code and report, must only be done in compliance with the policy set out in the course handbook.

© 2026 University of Cambridge

1 Cholesky Factorization with OpenMP on CSD3

In this coursework your task is to incrementally develop a high-performance CPU-based Cholesky Factorization routine exploiting OpenMP parallelization for running on a single node of a high performance computer such as CSD3.

Your routine should be written in C or C++. MPI or GPU acceleration must *not* be used.

For this coursework, showing evidence of a well-defined approach to performance testing is essential. You must describe the strategies for optimization and parallelization you are attempting, justify choices you make, providing evidence via performance data in figures and/or tables.

You are advised to manage your available time for the project wisely, not overly focussing on any one optimization/parallelization attempt.

Use tagging in your Git commit history to document and allow verification of your code and its changes at each significant attempt at optimization. Pick a simple naming convention for the tags that you can refer to in your report.

Mathematical Background

In the analysis of science data, it is often necessary to compute *covariance matrices*. For example, we might have n data points x_i, $i = 0, \ldots n-1$. A theory might then predict both the *mean* $\langle x_i \rangle$ and the *covariance* $\langle\langle x_i x_j \rangle\rangle \equiv \langle (x_i - \langle x_i \rangle)\,(x_j - \langle x_j \rangle)\rangle$ as a function of the model parameters. Here angle brackets denote expectation values of quantities inside the brackets for the model in question. Such covariance matrices C, with $C_{ij} = \langle\langle x_i x_j \rangle\rangle$, are *symmetric* and *positive-definite*.

It is often necessary to *invert* and/or compute the *determinant* of covariance matrices. To do this, computation of the *Cholesky factorization* of the covariance matrix C is a helpful step. Here one solves for a *lower-triangular* matrix (one whose entries above the leading diagonal are zero) L, such that

$$C = LL^{\mathrm{T}} \tag{1}$$

where the superscript T denotes the matrix transpose operation.

For example, if

$$C_2 = \begin{pmatrix} 4 & 2 \\ 2 & 26 \end{pmatrix} \tag{2}$$

then it may be verified that the lower-triangular matrix

$$L_2 = \begin{pmatrix} 2 & 0 \\ 1 & 5 \end{pmatrix} \tag{3}$$

satisfies Eq. (1).

The log of the determinant of C, i.e. $\log|C|$, is just then twice the sum of the log of the diagonal elements of L:

$$\log|C| = 2\sum_{p=0}^{n-1} \log L_{pp}. \tag{4}$$

© 2026 University of Cambridge

Function Interface

Your function should take as input a pointer to a double precision one-dimensional array containing the n^2 elements of C in row-major order and the integer n specifying the size of the matrix. It should be an *in-place* routine, overwriting the diagonal and lower-diagonal parts of C with the elements of L and the upper-diagonal part of C with the upper-diagonal part of L^{T}. So, were your routine to be called with a pointer to an array representing the example C_2 from Eq. (2) above, after completion the array would represent

$$\begin{pmatrix} 2 & 1 \\ 1 & 5 \end{pmatrix}. \tag{5}$$

On completion, your function should return, as a double precision number, the wall clock time elapsed in seconds in computing the Cholesky factorization.

A user should expect to be able to use your routine in their code after including a suitable header with a line like::

```
#include "mphil_dis_cholesky.h"
```

Baseline Implementation

To help you get started, the key loop of a functional implementation of Cholesky factorization is provided below:

```
for (int p=0; p<n; p++) { /* move along the diagonal of the matrix */
  double diag=sqrt(c[p*n+p]]);

  c[n*p+p]=diag;  /* update diagonal element */

  for (int j=p+1; j<n;j++){ /* update row to right of diagonal element */
    c[n*p+j]/=diag;
  }

  for (int i=p+1; i<n; i++){ /* update column below diagonal element */
    c[n*i+p]/=diag;
  }

  for (int j=p+1;j<n;j++){         /* update submatrix below-right */
    for (int i=p+1;i<n;i++){       /* of diagonal element */
      c[n*i+j]-=c[n*i+p]*c[n*p+j];
    }
  }

}
```

(TURN OVER)

© 2026 University of Cambridge

As long as the input matrix is indeed symmetric and positive-definite, the above algorithm should run to completion and be numerically stable. You should implement this first as a single-threaded program *and tag the commit* once you have it working.

You may follow the example code along using pencil and paper with the example C_2 and L_2 given above, with $n = 2$, to get a feel of how the algorithm works.

It may be seen that it takes $O(n^3)$ operations to compute the Cholesky factorization of an $n \times n$ matrix.

Optimization-related Coding Details

- You should optimize your implementation from the baseline to improve its runtime, even before attempting any parallelization.
- You may assume that your code will only be called with matrices with $n \leq 100000$. Implement a suitable check.
- You must use OpenMP to parallelize your code. For *every* line in your code at all related to OpenMP, you *must* add a comment that describes what the line is doing and your rationale for doing that. *Be sure to tag your first working parallel version of your code!*
- Include prominent comments in relevant files explaining what you are attempting to do/check/test etc. in that version of the code.
- Your code must not make use of external libraries other than OpenMP.

General Coding Details

- You should demonstrate good coding practice, as described in the C1 and C2 modules.
- Use a reasonable directory structure, providing source code for your library in the directory `src/` and a header file for users to include in the directory `include/` or similar.
- You should provide an example code illustrating the use of your library, in a directory `example/` or similar.
- You should include a `readme` file explaing how a user should compile your code into a library and how to use your routine. This file should also provide guidance about to use your routine in a performant manner.

Report Details

- Your report should give a brief description of your parallelization strategy, and give information via suitable plots and tables about performance scaling across matrix sizes and numbers of cores used. You should state which of the CSD3 partitions you used for each investigation.
- You should include a table describing the tagged commits of your code.

© 2026 University of Cambridge

- Make sure any performance data given comes with the code version, compilation options, the platform and number of threads used (if set at runtime) to allow it to be reproduced.

Other Assessment Criteria

- Correctness of your implementation may be assessed by comparing $\log|C|$ computed as in Eq. (4) from your returned L matrix against a reference value computed using well-established library code.
- You should provide a set of tests, using a `test/` directory.
- You should include example CSD3 submission scripts for running your example program.
- Both correctness and performance are important.

Other notes

- As the imagined use case of this project is for inclusion into another HPC codebase, containerization should not be used.
- The following code snippet might be helpful in populating a suitable array `c` with an example positive-definite matrix:

```
for (int i=0; i<n;i++){
  for (int j=0;j<n;j++){
      c[n*i+j] = corr(i,j,n);
  }
  c[n*i+i]=1.0;
}
```

where `corr()` is defined as:

```
double corr(double x, double y, double s){
  return .99*exp(-0.5*16.0*(x-y)*(x-y)/s/s);
}
```

END OF PAPER

© 2026 University of Cambridge